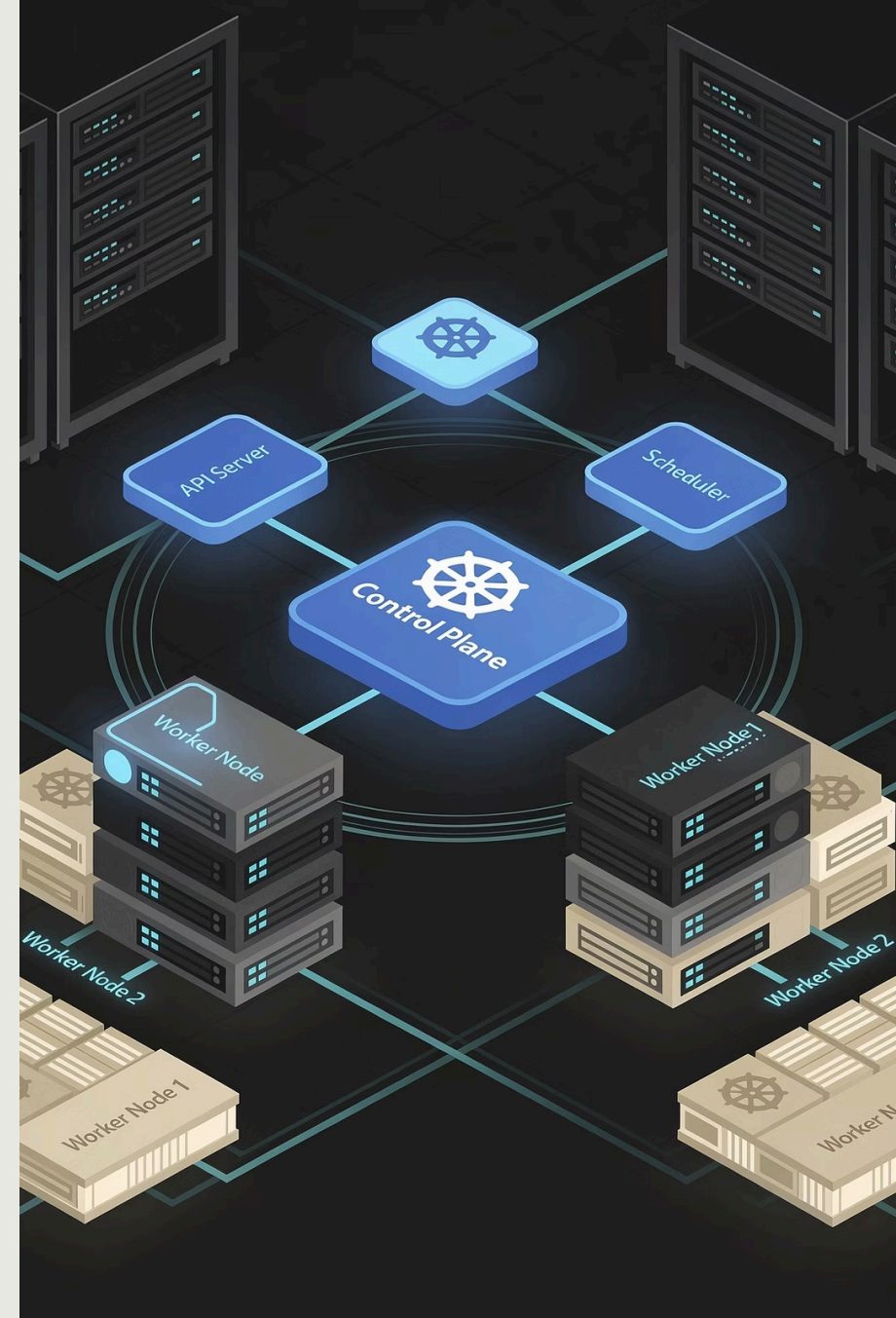


CONTROLLERS, REPLICASETS & DEPLOYMENTS

A deep-dive into Kubernetes controller architecture — from the reconciliation loop to rolling updates, owner references, and live debugging. Built for engineers preparing for the CKA and anyone who wants to understand what Kubernetes is *actually* doing under the hood.

CKA DEEP DIVE - RAJAGOPAL S

CONTROLLERS · REPLICASET · DEPLOYMENT



WHY YOU NEED TO UNDERSTAND THIS

THE GAP MOST ENGINEERS HAVE

Most engineers can *run* a Deployment. Very few can explain *why* it works. That gap becomes a liability when things break in production – and they will break. Understanding the controller layer is what separates a Kubernetes operator from a Kubernetes engineer.

WHAT THIS SESSION COVERS

01

WHY CONTROLLERS EXIST

The fundamental problem with bare Pods

02

CONTROL LOOP ARCHITECTURE

How Kubernetes reconciles desired vs. actual state

03

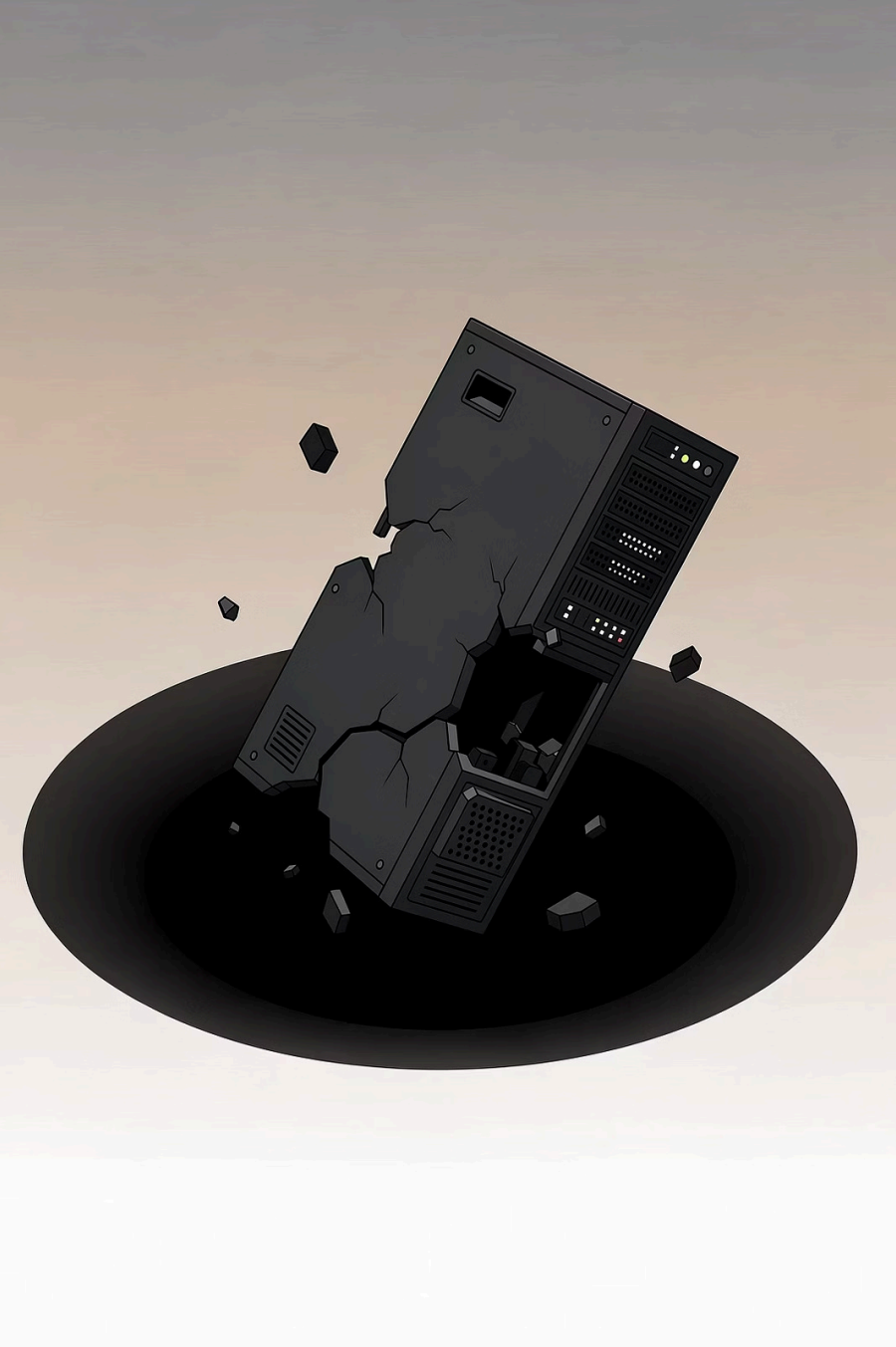
REPLICASET INTERNALS

Label selectors, adoption, and self-healing

04

DEPLOYMENT ROLLING UPDATES

Owner references, revision history, and rollback



PHASE 1 – THE PROBLEM WITH BARE PODS

Before controllers, start with this experiment. Run a Pod directly, then delete it – and watch it vanish forever.

CREATE THE POD

```
kubectl run nginx --image=nginx  
kubectl get pods
```

DELETE THE POD

```
kubectl delete pod nginx  
kubectl get pods
```

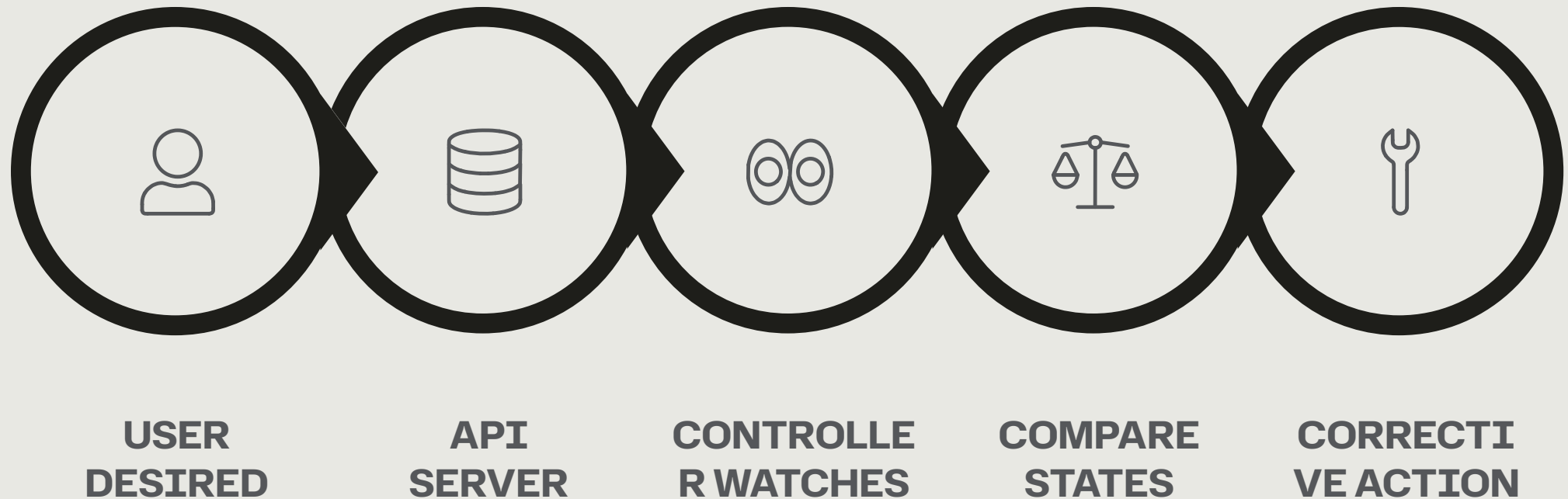
RESULT

Pod is **gone permanently**. No restart. No recovery. No trace.

A raw Pod has **no self-healing, no scaling, no version management, and no rollback capability**. It is a single ephemeral object. This is why Kubernetes introduces **Controllers** – to manage the lifecycle of Pods on your behalf.

PHASE 2 – THE KUBERNETES RECONCILIATION LOOP

Every Kubernetes controller is built on one elegant idea: continuously compare **desired state** against **actual state**, and take corrective action until they match. This is the **control loop** – the heartbeat of Kubernetes.



Example: if desired replicas = 3 and only 2 Pods are running, the controller immediately creates a third. This loop never stops – it is the mechanism behind self-healing, scaling, and rolling updates.

PHASE 3 – THE CONTROLLER MANAGER

WHAT IT IS

All controllers run as goroutines inside a single binary: `kube-controller-manager`. It runs on the control plane and hosts dozens of controllers simultaneously. Each controller watches the API Server for changes to its resource type.

VERIFY IT'S RUNNING

```
ps -ef | grep kube-controller-manager  
journalctl -u kube-controller-manager
```

CONTROLLERS INSIDE KUBE-CONTROLLER-MANAGER

REPLICASET

Maintains replica count

DEPLOYMENT

Manages rolling updates

NODE

Monitors node health

JOB

Runs tasks to completion

ENDPOINT

Populates Service endpoints

PHASE 4 – REPLICASET: DEEP CONCEPT

A **ReplicaSet** guarantees that a specified number of identical Pod replicas are always running. It does not track Pods by name – it uses **label selectors** to query the API Server and find matching Pods. If too few exist, it creates more. If too many exist, it deletes some.

CORE PROMISE

Exactly **replicas: N** Pods matching the selector will always be running, regardless of node failures or manual deletions.

LABEL-BASED SELECTION

ReplicaSet finds its Pods by querying: *"give me all Pods with label **app=web**"*. This is **not a name lookup** – it is a live label query executed on every reconciliation loop.

CONTROLLER HIERARCHY

ReplicaSet Controller watches API Server → detects desired vs. actual mismatch → creates or deletes Pod objects → Scheduler assigns them → Kubelet runs containers.

REPLICASET YAML – ANNOTATED

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx
```

KEY FIELDS EXPLAINED



REPLICAS: 3

Desired number of Pods. The controller enforces this count perpetually.



SELECTOR.MATCHLABELS

The label query used to find Pods. **Must match** the template labels exactly.



TEMPLATE

The Pod blueprint used when the controller needs to create new Pods. Includes labels that must match the selector.

PHASE 5 – INTERNAL REPLICASET WORKFLOW



KUBECTL APPLY

API SERVER

ETCD STORES REPLICASET

CONTROLLER WATCHES

When you run `kubectl apply -f replicaset.yaml`, the ReplicaSet object is persisted in etcd. The ReplicaSet Controller – running inside `kube-controller-manager` – receives a watch event, counts matching Pods, and creates exactly as many as needed to reach the desired replica count.

REPLICASET – VERIFICATION & SELF-HEALING DEMO

CHECK REPLICASET STATUS

```
kubectl get rs
kubectl describe rs nginx-rs
kubectl get pods --show-labels
```

EXPECTED OUTPUT

NAME	DESIRED	CURRENT	READY
nginx-rs	3	3	3

SELF-HEALING DEMO

Delete any Pod and watch the controller react:

```
kubectl delete pod <podname>
kubectl get pods -w
```

Within seconds, a **new Pod is created**. The ReplicaSet Controller detected the mismatch (desired=3, actual=2) and immediately created a replacement. This is the reconciliation loop in action.

An illustration of a desert landscape with rolling sand dunes in shades of beige and tan. A dark grey road curves from the bottom left towards the center. A black metal barrier with two vertical posts and a horizontal beam with yellow and black diagonal stripes stands across the road, blocking the path.

PHASE 6 – REPLICASET LIMITATION

ReplicaSet is powerful for maintaining replica count – but it has a critical gap: it **cannot safely perform rolling updates or version rollbacks**. If you change the image in a ReplicaSet spec, existing Pods are *not* replaced automatically. You would have to delete and recreate Pods manually, causing downtime.

✗ NO ROLLING UPDATE

Image changes do not propagate to running Pods automatically.

✗ NO ROLLBACK

No revision history is maintained. You cannot undo a change.

✗ NO DEPLOYMENT HISTORY

ReplicaSet is stateless about past versions.

✓ THE SOLUTION

Deployment – a higher-level controller that manages ReplicaSets and provides all the update and rollback capabilities that ReplicaSet lacks.

PHASE 7 – DEPLOYMENT ARCHITECTURE

THE THREE-LAYER HIERARCHY

PODS

Running containers on nodes

REPLICASET

Maintains desired Pod count

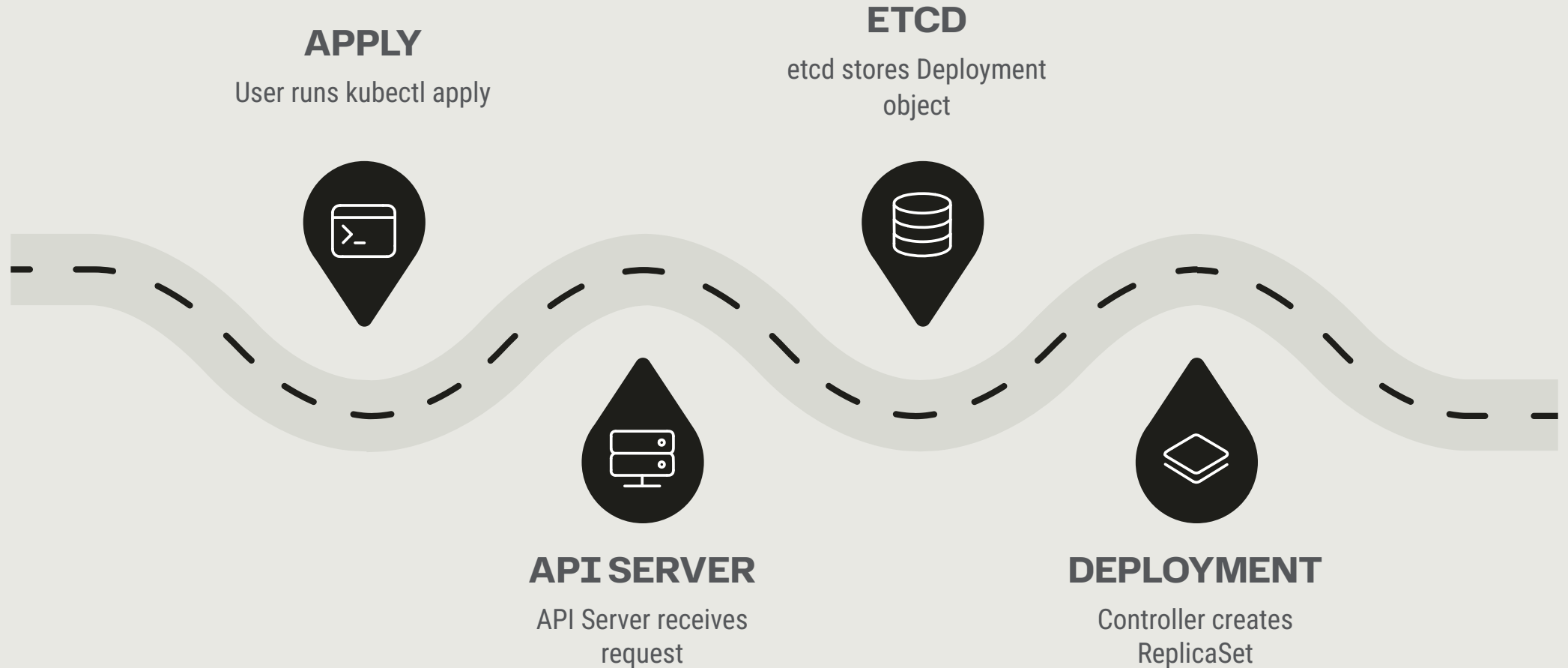
DEPLOYMENT

Manages ReplicaSets – orchestrates updates and rollbacks

DEPLOYMENT YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
```

PHASE 8 – DEPLOYMENT INTERNAL WORKFLOW



A Deployment never creates Pods directly. It creates a **ReplicaSet**, and the ReplicaSet creates Pods. This two-layer abstraction is what enables rolling updates – the Deployment simply manages which ReplicaSet is active and at what scale.

VERIFY THE DEPLOYMENT HIERARCHY

RUN THESE COMMANDS IN ORDER

```
kubectl get deployment  
kubectl get rs  
kubectl get pods
```

WHAT YOU'LL SEE

```
# ReplicaSet name contains Deployment name  
nginx-deploy-6b7d8c 3 3 3  
  
# Pod names contain ReplicaSet hash  
nginx-deploy-6b7d8c-abc12  
nginx-deploy-6b7d8c-def34  
nginx-deploy-6b7d8c-ghi56
```

READING THE NAMING CONVENTION

Kubernetes embeds the ownership hierarchy directly in resource names:

→ **NGINX-DEPLOY**

The Deployment name – the top of the hierarchy.

→ **NGINX-DEPLOY-6B7D8C**

ReplicaSet name = Deployment name + Pod template hash. Each unique template gets a unique hash.

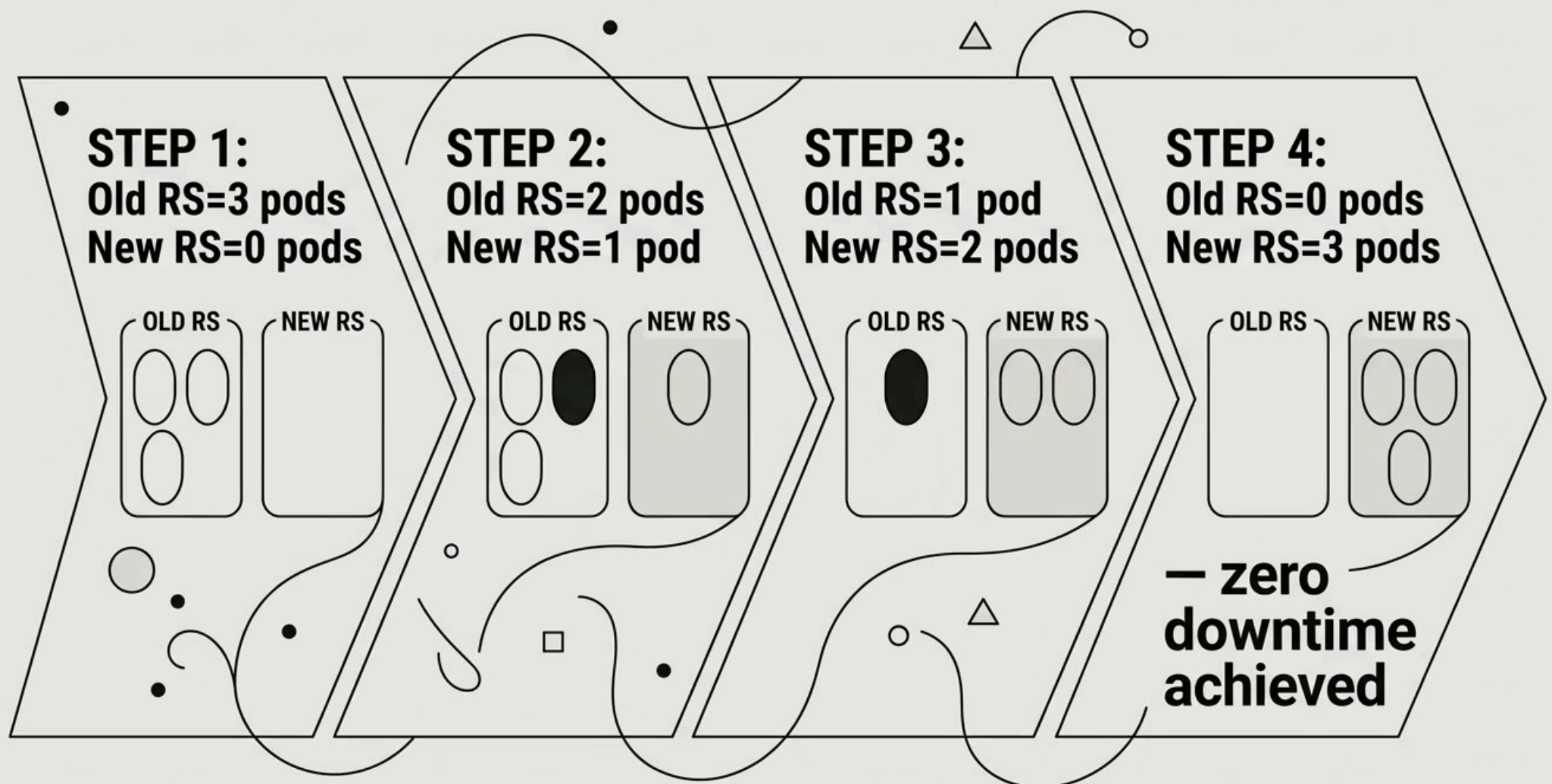
→ **NGINX-DEPLOY-6B7D8C-ABC12**

Pod name = ReplicaSet name + random suffix. Naming tells you exactly which ReplicaSet owns this Pod.

PHASE 9 – ROLLING UPDATE INTERNALS

When you update a Deployment's Pod template (e.g., change the container image), the Deployment Controller creates a **brand new ReplicaSet** and gradually scales it up while scaling down the old one. At no point does the application go offline.

```
kubectl set image deployment/nginx-deploy nginx=nginx:1.26
```



ROLLING UPDATE – VERIFY & CONTROL

WATCH IT LIVE

```
# Terminal 1 — watch pods
kubectl get pods -w
```

```
# Terminal 2 — watch ReplicaSets
kubectl get rs -w
```

```
# Terminal 3 — trigger update
kubectl set image \
  deployment/nginx-deploy \
  nginx=nginx:1.26
```

CHECK STATUS & HISTORY

```
kubectl rollout status \
  deployment nginx-deploy
```

```
kubectl rollout history \
  deployment nginx-deploy
```

CONTROL UPDATE BEHAVIOR

```
strategy:
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
```

maxUnavailable: max Pods that can be down during update. **maxSurge:** max extra Pods that can be created above desired count. Together they define the speed vs. safety tradeoff.

PHASE 10 – ROLLBACK & REVISION HISTORY

ROLLBACK TO PREVIOUS VERSION

```
# Undo last update
kubectl rollout undo \
  deployment nginx-deploy

# Rollback to specific revision
kubectl rollout undo \
  deployment nginx-deploy \
  --to-revision=1
```

The Deployment Controller **reactivates the old ReplicaSet** – it was never deleted, just scaled to zero. Rollback is nearly instantaneous because the Pods already have the right image cached.

REVISION HISTORY LIMIT

```
spec:
  revisionHistoryLimit: 5
```

Kubernetes retains old ReplicaSets (scaled to 0) up to this limit. Each represents one revision. Set too low and you lose rollback options. Set too high and you accumulate unused ReplicaSet objects in the cluster.

VIEW HISTORY

```
kubectl rollout history \
  deployment nginx-deploy
```

REVISION CHANGE-CAUSE

```
1 nginx:1.24
2 nginx:1.25
3 nginx:1.26
```

REPLICASET VS. DEPLOYMENT – SIDE BY SIDE

Understanding when Kubernetes uses each resource – and why you should almost never create a ReplicaSet directly.

Feature	ReplicaSet	Deployment
Primary Purpose	Maintain a fixed replica count	Manage full application lifecycle
Rolling Updates	✗ Not supported	✓ Automatic, configurable
Rollback	✗ No history	✓ Full revision history
Update Strategy	Manual pod deletion required	maxUnavailable / maxSurge
Typical Usage	Rarely used directly	Standard for all stateless apps
Manages	Pods (via labels)	ReplicaSets (which manage Pods)

📌 In production, you should almost always use a **Deployment**. Only create a ReplicaSet directly if you need fine-grained control over a specific replica set without update management – which is extremely rare.

OWNER REFERENCES – THE RESOURCE HIERARCHY MECHANISM

WHAT ARE OWNER REFERENCES?

Kubernetes stores **parent-child relationships** inside `metadata.ownerReferences` on every controlled resource. This is how the system knows which controller owns which resource – and what to clean up when a parent is deleted.

```
metadata:
  ownerReferences:
    - apiVersion: apps/v1
      kind: ReplicaSet
      name: nginx-deploy-6b7d8c
```

VERIFY OWNER REFERENCES LIVE

```
# Inspect a Pod
kubectl get pod <pod-name> -o yaml
# Look for ownerReferences → ReplicaSet
```

```
# Inspect the ReplicaSet
kubectl get rs <rs-name> -o yaml
# Look for ownerReferences → Deployment
```

CASCADING DELETION

When you `kubectl delete deployment nginx-deploy`, Kubernetes follows the owner reference chain: Deployment deleted → ReplicaSets deleted → Pods deleted. This is called **garbage collection** and is entirely driven by `ownerReferences`.

LABEL SELECTORS – HOW REPLICASET FINDS ITS PODS

ReplicaSet does **not** maintain a list of Pod names. Every reconciliation loop, it queries the API Server: *"Find all Pods with label app=web"*. This means labels are not cosmetic – they are the **control mechanism**.



LABEL MATCHING

Only Pods whose labels match the ReplicaSet's `selector.matchLabels` are counted and managed. A Pod with `app=db` is completely invisible to a ReplicaSet selecting `app=web`.



POD ADOPTION

If a matching Pod exists without an owner, the ReplicaSet **adopts** it by writing itself into the Pod's `ownerReferences`. This is not a side effect – it is by design.

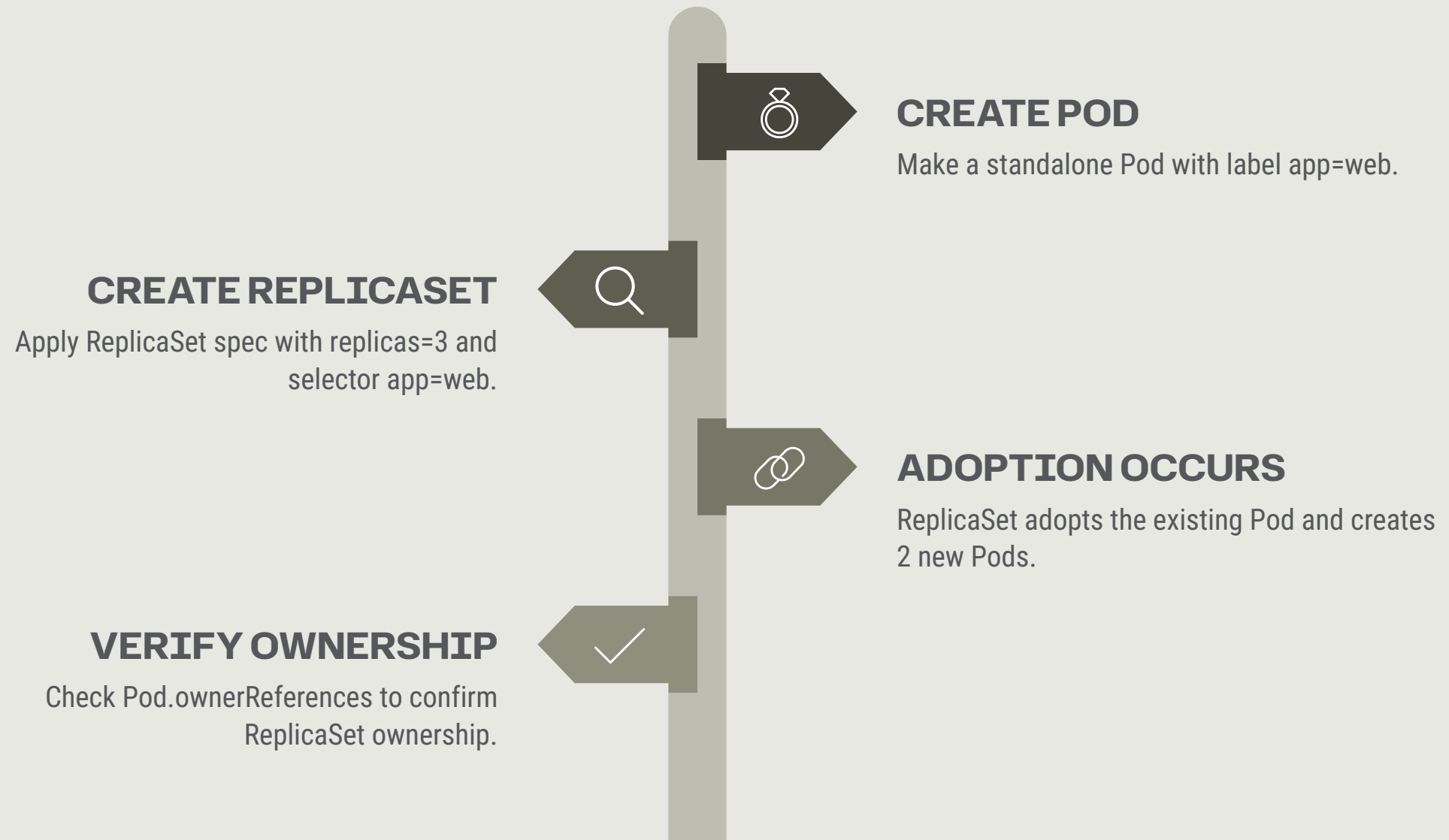


LABEL CONFLICTS IN PRODUCTION

Two controllers using the same label selector will fight over the same Pods – causing unpredictable scaling behavior. **Label design is a critical architectural decision.**

DEMO – REPLICASET POD ADOPTION

This hands-on experiment makes the reconciliation loop and label selector mechanism **visually undeniable**. You will create an unmanaged Pod, then watch a ReplicaSet claim ownership of it – without you doing anything.



This experiment reveals a profound truth about Kubernetes: **controllers are reactive, not prescriptive**. The ReplicaSet does not care who created the Pod – it cares only whether the label matches and whether the desired count is satisfied.

ADOPTION DEMO – STEP BY STEP

STEP 1 – CREATE UNMANAGED POD

```
kubectl run manual-pod \
  --image=nginx \
  --labels app=web

kubectl get pod manual-pod -o yaml
# ownerReferences: [] — no controller!
```

STEP 2 – CREATE REPLICASET (REPLICAS=3)

```
vi rs-demo.yaml
# replicas: 3, selector: app=web

kubectl apply -f rs-demo.yaml
kubectl get pods
```

WHAT YOU'LL OBSERVE

The ReplicaSet creates only **2 new Pods** – not 3. It counted `manual-pod` as an existing match and adopted it.

NAME	READY	STATUS
manual-pod	1/1	Running ← adopted!
web-rs-abc12	1/1	Running
web-rs-def34	1/1	Running

STEP 3 – VERIFY ADOPTION

```
kubectl get pod manual-pod -o yaml
# ownerReferences:
#   kind: ReplicaSet
#   name: web-rs
```

The Pod now has an owner. The ReplicaSet wrote itself into `ownerReferences`. This is **Controller Adoption**.

ADOPTION DEMO – LABEL REMOVAL EXPERIMENT

This next step makes the label selector concept impossible to misunderstand. Remove the `app=web` label from a running Pod and watch the controller react in real time.

REMOVE THE LABEL

```
# Pick any managed Pod
kubectl get pods --show-labels

# Remove the app label (note the minus sign)
kubectl label pod <podname> app-

# Watch what happens
kubectl get pods -w
```

WHAT HAPPENS NEXT



POD BECOMES ORPHAN

The relabeled Pod no longer matches the selector. ReplicaSet stops managing it – but does not delete it.



REPLICASET DETECTS SHORTFALL

Matching Pods drop from 3 to 2. Reconciliation loop fires immediately.



NEW POD CREATED

A brand new Pod with the correct label is created to restore desired count = 3. You now have 4 Pods total – 3 managed, 1 orphan.

POWER DEMO

ROLLING UPDATE VISUALIZATION DEMO

This is the definitive way to understand rolling updates. Instead of reading about ReplicaSet transitions, you will **watch them happen live** across three terminal windows simultaneously. Every Kubernetes engineer should run this at least once.

TERMINAL 1

```
kubectl get pods -w
```

Watch Pods terminate and start in real time

TERMINAL 2

```
kubectl get rs -w
```

Watch old RS scale down as new RS scales up

TERMINAL 3

```
kubectl set image \
  deployment/nginx-deploy \
  nginx=nginx:1.25
```

Trigger the rolling update and observe the cascade

ROLLING UPDATE DEMO – FULL WALKTHROUGH

SETUP – DEPLOY NGINX:1.24

```
kubectl create deployment nginx-deploy \  
  --image=nginx:1.24 \  
  --replicas=3
```

```
kubectl get deployment  
kubectl get rs  
kubectl get pods
```

TRIGGER UPDATE TO NGINX:1.25

```
kubectl set image \  
  deployment/nginx-deploy \  
  nginx=nginx:1.25
```

OBSERVE TWO REPLICASETS

```
kubectl get rs
```

NAME	DES	CUR	RDY	
nginx-deploy-6b7d8c	1	1	1	← old
nginx-deploy-7f8g9h	2	2	2	← new

FINAL STATE AFTER UPDATE

```
kubectl get rs
```

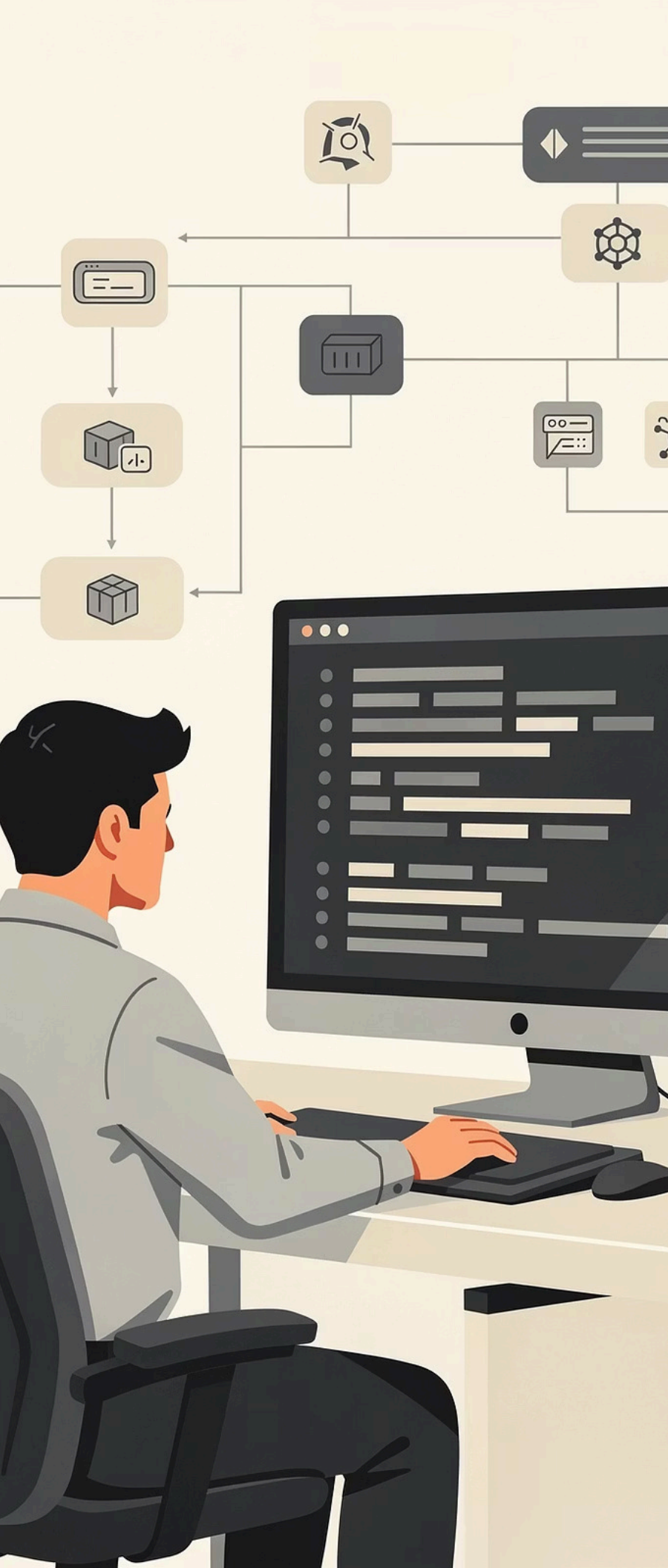
NAME	DES	CUR	RDY	
nginx-deploy-6b7d8c	0	0	0	← idle
nginx-deploy-7f8g9h	3	3	3	← active

The old ReplicaSet is **kept at zero** – not deleted. This is your rollback safety net. If you run `kubectl rollout undo`, the old RS is simply scaled back up.

CONFIRM SUCCESSFUL ROLLOUT

```
kubectl rollout status \  
  deployment nginx-deploy
```

Output:
deployment "nginx-deploy" \
successfully rolled out



PHASE 11 – CKA-LEVEL DEBUGGING FLOW

When a Deployment or rollout fails in a real cluster, a structured investigation order prevents wasted time. Always start at the highest abstraction level and work down – the failure is usually visible at exactly one layer.

1 INSPECT THE DEPLOYMENT

```
kubectl describe deployment nginx-deploy
```

Look for: failed conditions, unavailable replicas, bad strategy config.

2 CHECK REPLICASETS

```
kubectl get rs
kubectl describe rs <rs-name>
```

Look for: stuck ReplicaSets, unexpected desired counts, selector mismatches.

3 INSPECT FAILING PODS

```
kubectl describe pod <pod-name>
kubectl logs <pod-name>
```

Look for: image pull errors, OOMKilled, CrashLoopBackOff, failed readiness probes.

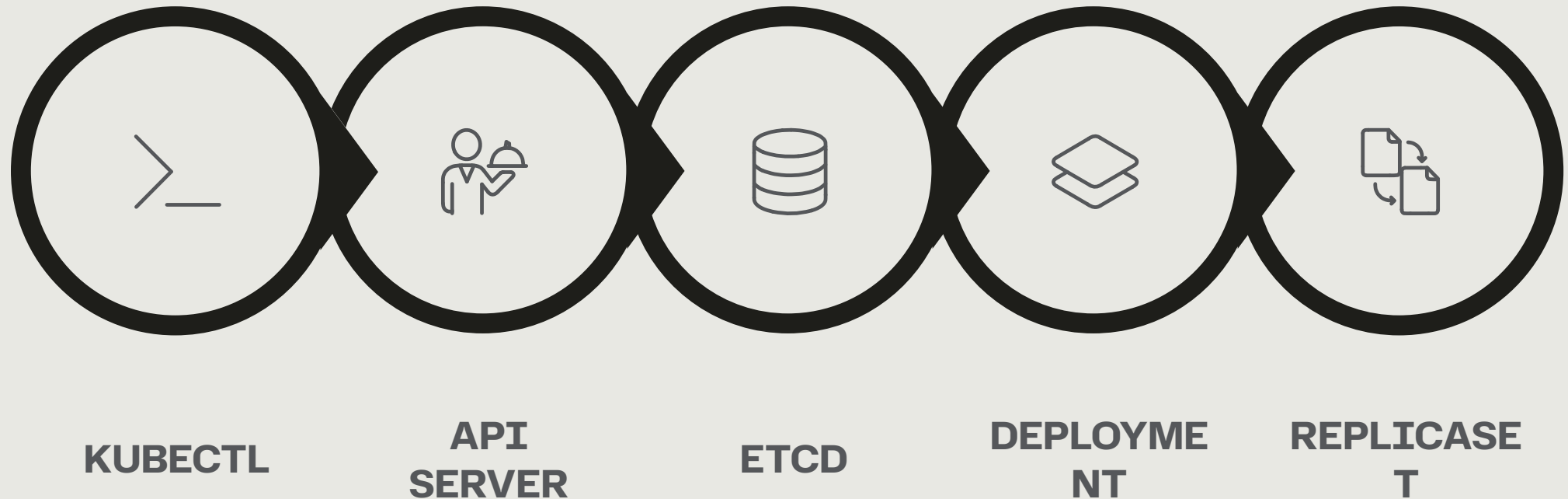
4 CHECK CLUSTER EVENTS

```
kubectl get events \
  --sort-by='.lastTimestamp'
```

Events reveal scheduling failures, quota violations, and controller errors not visible elsewhere.

COMPLETE CONTROLLER ARCHITECTURE – END TO END

Every request you make flows through this exact chain. Understanding each layer tells you exactly where to look when something breaks.



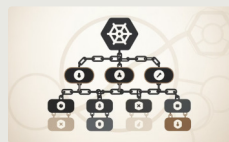
Every component in this chain is independently observable. If any step fails, the subsequent steps never execute – making it straightforward to isolate the fault layer using `kubectl describe` and `kubectl get events`.

KEY CONCEPTS – COMPLETE REFERENCE



RECONCILIATION LOOP

The core mechanism of every Kubernetes controller. Perpetually compares desired state (stored in etcd) against actual state (observed cluster) and takes corrective action.



OWNER REFERENCES

Kubernetes embeds parent-child relationships in `metadata.ownerReferences`. Enables cascading deletion, adoption, and garbage collection – all automatic.



LABEL SELECTORS

Controllers do not track Pods by name. They query by labels. This enables adoption, dynamic reconciliation, and decoupled resource management. Label design is an architectural decision.



ROLLING UPDATE

Deployment manages two ReplicaSets during an update – scaling up the new one while scaling down the old. `maxUnavailable` and `maxSurge` control the speed and safety of the transition.

WHAT'S NEXT – THE CONTROLLER FAMILY

Controllers, ReplicaSets, and Deployments are built on the same **control loop principle**. Once you understand this foundation, every other Kubernetes workload resource follows the same pattern.

1

STATEFULSET

Like a Deployment, but for stateful applications. Pods get stable network identities, ordered startup/shutdown, and persistent volume claims that survive Pod restarts.

2

DAEMONSET

Ensures exactly one Pod runs on every node (or a selected subset). Used for node-level infrastructure: log collectors, monitoring agents, network plugins.

3

JOBS & CRONJOBS

Runs Pods to completion rather than keeping them alive. Jobs for one-time tasks; CronJobs for scheduled recurring workloads. Same reconciliation loop, different desired state definition.

4

CONFIGMAP & SECRETS

Configuration injection into Pods – decoupling application code from environment-specific configuration. Critical for building portable, environment-agnostic workloads.

SESSION TAKEAWAYS

THE MENTAL MODEL

Everything in Kubernetes is driven by the same loop: **watch** → **compare** → **act**. Once this is internalized, no Kubernetes behavior will ever surprise you again. Controllers are not magic – they are persistent, opinionated reconcilers.

THE DEBUGGING MINDSET

Always start at the **highest abstraction layer** (Deployment) and work down (ReplicaSet → Pod → Events). The failure is visible at exactly the layer where the chain breaks.

NEVER CREATE BARE PODS IN PRODUCTION

Always use a Deployment.

NEVER CREATE REPLICASETS DIRECTLY

Let Deployment manage them.

LABELS ARE CONTROL, NOT METADATA

Design them carefully.

OLD REPLICASETS = ROLLBACK SAFETY NET

Set revisionHistoryLimit wisely.

OWNERREFERENCES EXPLAIN EVERYTHING

Inspect with `-o yaml` first.